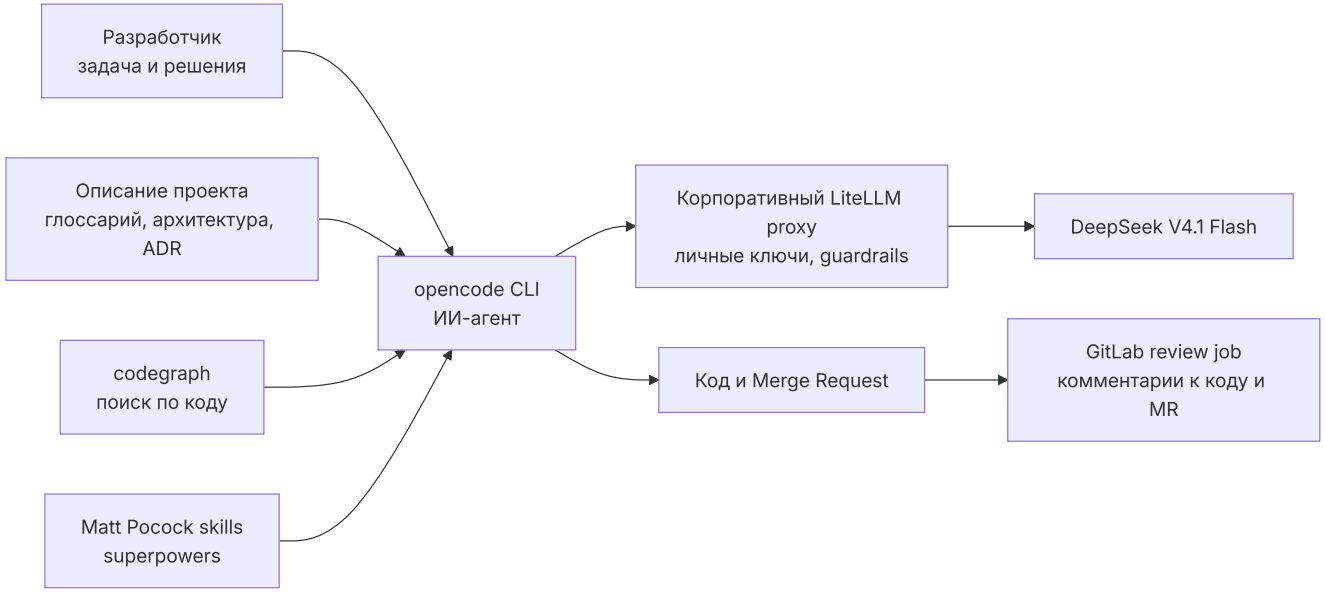
Как мы работаем с ИИ-агентами при разработке

Мы ведём задачи через **конвейер фаз**: на каждом этапе определяем вход, получаем результат и проверяем его. Так задача проходит путь от идеи до кода с понятными точками контроля для разработчика.



Инструменты и контекст. Разработчик работает с агентом через opencode, CLI-приложение для сессий разработки, скиллов и инструментов. Основная модель — **DeepSeek V4.1 Flash**. Запросы к ней идут через корпоративный LiteLLM proxy. У каждого сотрудника отдельный ключ доступа к прокси: по нему сохраняется история всех обращений к модели и учитывается расход токенов. Защитные фильтры (guardrails) на прокси маскируют ПДн, токены, секреты и ссылки на внутренние ресурсы до отправки запроса в модель.

Агент использует текстовое описание проекта: глоссарий, архитектурные правила и принятые решения. Поиск по коду выполняем с помощью codegraph. Для повторяемых этапов используем Matt Pocock skills и superpowers.

Дополнительные инструменты. Perplexity помогает искать внешнюю информацию и сверять источники. В Langfuse мы смотрим трассировки обращений к модели, задержки и расход токенов.

Как проходит задача:

1. **Разобраться в задаче и контексте.** Уточняем требования, находим нужные части проекта, фиксируем термины и решения.
2. **Спроектировать и описать.** Составляем спецификацию (`PRD`). Если нужен ручной контроль, делим её на небольшие тикеты с зависимостями.
3. **Реализовать.** Либо запускаем `implement` по одному тикету, либо составляем план через `writing-plans` и выполняем его через `subagent-driven-development` . Агент пишет код небольшими проверяемыми шагами.
4. **Проверить.** Прогоняем тесты и проводим ревью после каждой задачи, исправляем замечания и открываем Merge Request.

Ревью в GitLab. В собственном экземпляре GitLab отдельная CI-джоба анализирует MR. Она оставляет комментарии к строкам кода и общий комментарий ко всему MR. Пока автоматическое ревью не всегда даёт полезные замечания. Разработчик проверяет их и решает, какие правки вносить.